**Universidade Estadual Vale do Acaraú - UVA**
**Curso:** Ciências da Computação
**Disciplina:** Construção e Análise de Algoritmos
**Professor:** Cláudio Carvalho

## Lista 03 - Programação Dinâmica

1. Algumas aplicações de biologia precisam comparar o DNA de dois ou mais organismos para ver o grau de similaridade entre eles. Uma cadeia de DNA consiste em uma sequência de bases - *adenina*, *timina*, *guanina*, *citosina* - representadas por $A$, $T$, $G$ e $C$, respectivamente. Proponha um algoritmo de complexidade $O(mn)$ que, dadas duas cadeias de DNA com tamanhos $m$ e $n$, diga o tamanho da maior subcadeia comum a elas. Por exemplo, o tamanho da maior subcadeia comum a $ACC$GGTCG$A$GTGC$G$CGG$AAG$ e G$C$GT$T$CGG$AA$TGCCG$TT$G$C$ é 12.

2. Um *palíndromo* é uma sequência de caracteres que é igual ao seu reverso. Na sequência ACGT$A$G$T$CA$AAATCG$, os trechos destacados formam um palíndromo. Proponha um algoritmo de complexidade $\Theta(n^2)$ que receba uma sequência $S$ de tamanho $n$ e retorne o tamanho da sua maior subsequência que seja um palíndromo.

3. Você fará uma viagem longa (iniciada no $Km$ 0). No percurso, há $n$ hotéis, cujas distâncias a partir do início são $a_1 < a_2 < \ldots < a_n$. Você deve fazer uma parada por dia, e só pode parar nesses hotéis. A viagem termina no Km $a_n$. Você deseja viajar $200km$ por dia, mas nem sempre é possível. Viajando $X$ $Km$ por dia, você recebe uma penalização de $|200 - X|$. Escreva um algoritmo de complexidade $O(n^2)$ que, dada uma sequência de $n$ paradas, diga a penalização mínima que você receberá.

4. Um *segmento* de um vetor $A[1 \ldots n]$ é um subvetor $A[i \ldots k]$, com $1 \leq i \leq k \leq n$. A condição $i \leq k$ garante que o segmento não é vazio. Seja a *solidez* de um vetor a maior soma de um segmento deste. Dado um vetor $A$ com $n$ valores inteiros, proponha um algoritmo de de complexidade $O(n)$ que retorne a sua solidez.
   **Ex.:** A solidez de $A = \langle 5\ 15\ -30$ 10 -5 40 10 $-5\ 3\rangle$ é 55.

5. Você deseja cortar um pedaço de madeira em vários pedaços. Suponha uma máquina que faça apenas um corte de cada vez, e que o custo do corte é igual ao tamanho do pedaço de madeira que é posto na máquina. Observe que a sequência dos cortes faz diferença no custo. Por exemplo, considere um pedaço de madeira de 10 metros que deve ser cortado nos pontos 2, 4 e 7. Se os cortes forem feitos na ordem $2, 4, 7$, o custo será 24; se forem feitos na ordem $4, 2, 7$, o custo sera 20. Proponha um algoritmo de programação dinâmica que, dados um pedaço de madeira de comprimento $L$ e uma sequência de $k$ pontos de corte, diga o menor custo para efetuar os cortes.

6. Dados um pedaço de madeira de tamanho $n$ e duas sequências de $k$ valores $A = \langle a_1, \ldots a_k \rangle$ e $B = \langle b_1, \ldots, b_k \rangle$, indicando que o preço de um pedaço de madeira de tamanho $a_i$ é $b_i$, $1 \leq i \leq k$. Proponha um algoritmo de complexidade $O(n^2)$ que diga o maior lucro que se pode obter com o pedaço de madeira dado.

7. Dada uma sequência de inteiros $A = \langle a_1, a_2, \ldots, a_n \rangle$, dizemos que $S$ é uma subsequência de $A$ de tamanho $k$ se está na forma: $\langle a_{i_1}, a_{i_2}, \ldots, a_{i_k} \rangle$, onde $1 \leq i_1 < i_2 < \ldots \leq i_k$. $S$ é crescente se $a_{i_1} < a_{i_2} < \ldots < a_{i_n}$. Proponha um algoritmo de programação dinâmica que retorne o tamanho da maior subsequência crescente de uma sequência dada. Por exemplo, para $A = \langle 5, 2, 8, 6, 3{,}5, 9, 6{,}8 \rangle$, o algoritmo deve retornar o valor 5.

8. A *Distância de Edição* entre duas sequências pode ser vista como o menor número de edições (inserções, remoções e substituições de caracteres) necessárias para deixá-las iguais. Por exemplo, a distância de edição entre $SNOWY$ e $SUNNY$ é 3 (a inserção do $U$, a troca do $O$ por $N$, e a remoção do $W$). Proponha um algoritmo de programação dinâmica que receba duas sequências e calcule a distância de edição entre elas.

9. Uma balsa leva carros de um lado para o outro de um rio. A balsa tem duas pistas, cada uma com tamanho $L$. Os carros que devem ser embarcados na balsa estão em fila (ou seja, um carro só é embarcado quando chega à frente da fila. A fila tem $n$ carros $\langle c_1, c_2, \ldots, c_n \rangle$ com tamanhos $\langle t_1, t_2, \ldots, t_n \rangle$. Queremos colocar o maior número de carros na balsa, decidindo em que faixa cada carro deve ficar. Proponha um algoritmo de programação dinâmica que resolva esse problema.

   **Dica:** Use uma matriz $M[k, A, B]$ que indica o máximo de carros que podem ser embarcados, dados a fila $\langle c_k, \ldots, c_n \rangle$, a pista 1 com tamanho $A$, e a 2 com tamanho $B$.

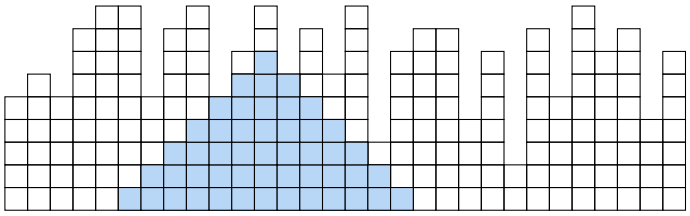
10. Considere o problema de arrumar a impressão de um parágrafo com uma fonte de tamanho fixo. O texto de entrada é uma sequência de $n$ palavras com tamanhos $t_1, \ldots, t_n$ caracteres. Queremos imprimir este parágrafo de forma arrumada, considerando que cada linha comporta no máximo $m$ caracteres. Se uma dada linha contém palavras $w_i, \ldots, w_j$, com $i \leq j$, e temos exatamente um espaço entre as palavras, o número de caracteres de espaço extras no final de cada linha é $m - j + i + t_i + \ldots + t_j$, que deve ser não negativo. Chamamos de *defeito* a soma dos cubos dos espaços a final de cada linha. Proponha um algoritmo de programação dinâmica que calcule o *defeito mínimo* que pode ser obtido com um conjunto de palavras.

11. Proponha um algoritmo que resolva o problema *3-Partição* descrito a seguir. Dada uma coleção de inteiros $X = \langle x_1, \ldots, x_n \rangle$, determinar se é possível fazer uma partição destes valores em três grupos disjuntos $I$, $J$ e $K$ tais que:

    $$\sum_{x_i \in I} x_i = \sum_{x_j \in J} x_j = \sum_{x_k \in K} x_k = \frac{1}{3} \times \sum_{i=1}^{n} x_i.$$

    **Exemplo:** $X = \langle 1, 2, 3, 4, 4, 5, 8 \rangle$, é possível fazer $I = \langle 1, 8 \rangle$, $J = \langle 4, 5 \rangle$ e $K = \langle 2, 3, 4 \rangle$.

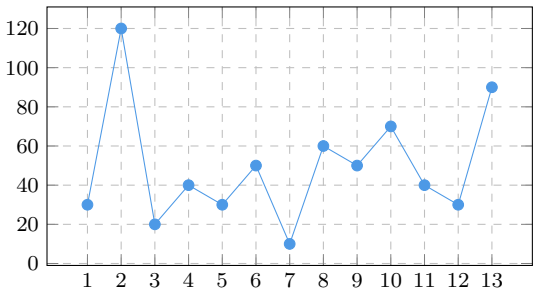
12. (Maratona de Programação da SBC - 2016) Dois irmãos estavam brincando com cubos de madeira e queriam construir um muro, que acabou ficando incompleto, com colunas tendo diferentes alturas, como na figura a seguir:



Eles decidiram que a brincadeira agora seria retirar os cubos das colunas (de cima para baixo), de tal maneira que, no final, reste apenas um triângulo isósceles (nos moldes do que está destacado). Os cubos não podem ser recolocados nas colunas do muro. Proponha um algoritmo de complexidade $O(n)$ que, dada a sequência com as alturas de $n$ colunas, retorne a altura do maior triângulo isósceles que pode ser obtido.
**Exemplo:** Para a sequência de colunas da figura acima, a resposta é 7.

13. Você deseja investir na bolsa de valores. Como não tem experiência, selecionou uma única empresa e acompanhou os valores diários de sua ações durante $n$ dias, e ficou curioso para saber o quanto teria ganho se tivesse feito um investimento nas ações dessa empresa nesse período. Você decidiu nunca ter mais do que uma ação simultaneamente. A corretora de valores cobra uma taxa fixa a cada compra de ação. Proponha um algoritmo que, dados a quantidade $n$ de dias, a taxa da corretora e a sequência de valores de ações nos $n$ dias, indique o lucro máximo que pode ser obtido investindo no período, podendo inclusive não investir. Analise a complexidade do seu algoritmo.

No gráfico a seguir, estão representados os valores das ações para um período de 13 dias. Supondo que a taxa para compra de ações é 30, o lucro máximo que pode ser obtido nesse período é 120 (ver quadro à direita).




Taxa da corretora = 30,00.

| Compra | | Venda | | Lucro |
|---|---|---|---|---|
| **dia** | **valor** | **dia** | **valor** | |
| 01 | 30,00 | 02 | 120,00 | 60,00 |
| 07 | 10,00 | 10 | 70,00 | 30,00 |
| 12 | 30,00 | 13 | 90,00 | 30,00 |
| | | | **Total** | **120,00** |

14. Dados um fornecimento ilimitado de moedas de valores $M = \{m_1, \ldots, m_n\}$ e um valor $v$, proponha um algoritmo de complexidade $O(nv)$ que diga se é possível pagar exatamente o valor $v$ com as moedas existentes.
**Exemplo:** Para $M = \{5,\ 10\}$, não é possível pagar 12, mas é possível parar 35.

15. Dados um conjunto $A$ de inteiros positivos e um inteiro $T$, proponha um algoritmo de complexidade $O(nT)$ que diga se há um subconjunto de $A$ cuja soma seja $T$.
**Exemplo:** Para $A = \{2, 5, 7, 8\}$ e $T = 11$, a resposta é **Não**.

16. Considere o jogo em que um mediador produz uma sequência $s_1, \ldots, s_n$ de cartas ($n$ é par), cada carta $s_i$ tem valor $v_i$, $1 \leq i \leq n$. Em seguida, dois jogadores devem pegar uma carta, mas apenas a primeira ou a última carta da sequência restante podem ser escolhidas. O jogador vencedor é o que obtiver a maior soma de cartas. Proponha um algoritmo que calcule a menor soma que o jogador 1 pode obter para não perder.

17. Considere o jogo em que são colocadas algumas pedras em uma mesa, e cada jogador pode pegar 1, 3 ou 8 pedras de cada vez. O jogador que pegar as últimas pedras é o vencedor. Dada uma quantidade $n$ de pedras, proponha um algoritmo de complexidade $O(n)$ que diga se é possível o primeiro jogador vencer o jogo. Por exemplo, com 6 pedras na mesa, ele sempre perde. As possíveis sequências de retiradas são:
$\langle 1, 1, 1, 1, 1, 1\rangle$, $\langle 1, 1, 1, 3\rangle$, $\langle 1, 1, 3, 1\rangle$, $\langle 1, 3, 1, 1\rangle$, $\langle 3, 1, 1, 1\rangle$ e $\langle 3, 3\rangle$.

18. Dada uma sequência de $n$ matrizes $M = M_1, M_2, \ldots, M_n$ de dimensões, respectivamente, $m_0 \times m_1, m_1 \times m_2, \ldots, m_{n-1} \times m_n$. Proponha um algoritmo que diga o menor número de multiplicações para calcular o produto das matrizes na sequência dada.

**Observações:**

- a multiplicação de matrizes não é comutativa ($A \times B \neq B \times A$), mas é associativa ($A \times (B \times C) = (A \times B) \times C$);
- a multiplicação de uma matriz de ordem $m \times n$ por outra de ordem $n \times p$ exige $m \times n \times p$ operações de multiplicação, e resulta em uma matriz de ordem $m \times p$.

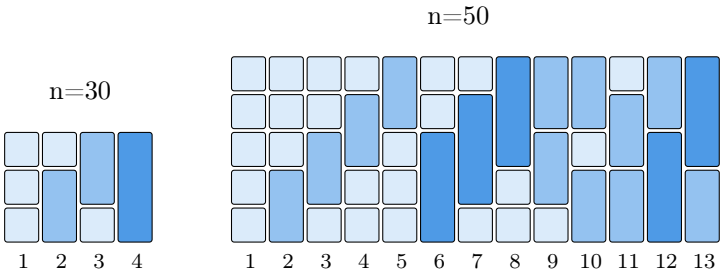
**Exemplo:** A sequência $\langle 200, 2, 30, 20, 5\rangle$ indica que são quatro matrizes de ordens $200 \times 2$, $2 \times 30$, $30 \times 20$ e $20 \times 5$, respectivamente. Algumas formas de fazer o produto delas são mostradas no quadro a seguir:

| **Ordem de Multiplicação** | **Cálculo do Custo** | | | | | **Custo** |
|---|---|---|---|---|---|---|
| $(((M_1 \times M_2) \times M_3) \times M_4)$ | $200 \times 2 \times 30$ | $+$ | $200 \times 30 \times 20$ | $+$ | $200 \times 20 \times 5$ | 152.000 |
| $((M_1 \times M_2) \times (M_3 \times M_4))$ | $200 \times 2 \times 30$ | $+$ | $30 \times 20 \times 5$ | $+$ | $200 \times 30 \times 5$ | 45.000 |
| $(M_1 \times ((M_2 \times M_3) \times M_4))$ | $2 \times 30 \times 20$ | $+$ | $2 \times 20 \times 5$ | $+$ | $200 \times 2 \times 5$ | 3.400 |
| $((M_1 \times (M_2 \times M_3)) \times M_4)$ | $2 \times 30 \times 20$ | $+$ | $200 \times 2 \times 20$ | $+$ | $200 \times 20 \times 5$ | 29.200 |
| $(M_1 \times (M_2 \times (M_3 \times M_4)))$ | $30 \times 20 \times 5$ | $+$ | $2 \times 30 \times 5$ | $+$ | $200 \times 2 \times 5$ | 5.300 |

19. Dada uma sequência $A$ de valores inteiros; uma subsequência contígua $A$ é dita nula, se não for vazia e a soma de seus elementos for 0. Por exemplo, se $A = \langle 2, -1, 0, -1, 1\rangle$, são subsequências nulas: $\langle 2, -1, 0, -1\rangle$, $\langle 0\rangle$, $\langle 0, -1, 1\rangle$ e $\langle -1, 1\rangle$. Dada uma sequência $A = \langle a_1, \ldots, a_n\rangle$, onde $-9 \leq a_i \leq 9$ para $1 \leq i \leq n$, proponha um algoritmo de complexidade $O(n)$ que diga o número de subsequências nulas desta.

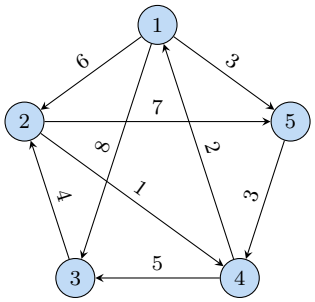
20. Dada uma cadeia $A$, formada exclusivamente por $n$ dígitos decimais, proponha um algoritmo que determine em tempo $O(n)$ a quantidade de intervalos do tipo $[i, j]$, com $1 \leq i \leq j \leq n$, tal que a subcadeia $A[i \ldots j]$ seja um múltiplo de 3. Por exemplo, $A = 101012$ possui 5 intervalos desse tipo. São eles: $[1, 5]$, $[2, 2]$, $[4, 4]$, $[4, 6]$ e $[5, 6]$.

21. João dispõe de três tipos de azulejos com dimensões $10 \times 10$, $20 \times 10$ e $30 \times 10$. Ele decidiu que irá dispor as peças em fileiras de dimensões de $10 \times n$, onde $n$ é múltiplo de 10, de modo que não haja duas fileiras com uma mesma sequência de peças. Elabore um algoritmo que determine em tempo linear o número máximo de fileiras que João conseguirá fazer. A figura abaixo mostra as possibilidades para $n = 30$ e $n = 50$.




22. Dado um digrafo ponderado $D = (V, A, w)$, onde $V$ é um conjunto de vértices, e $A$ é um conjunto de arcos, e $w : A \to \mathbb{R}^*_+$. Seja $a = (x, y)$ um arco de $A$, então $w(a)$ é o tamanho do caminho que liga $x$ diretamente a $y$ (se não há tal caminho, $w(a) = \infty$). A distância entre dois vértices $x$ e $y$ é o tamanho do menor caminho direcionado de $x$ a $y$. Proponha um algoritmo que calcule, em tempo $O(n^3)$ (onde $n = |V|$), a distância entre todos os pares de vértices de $D$. Por exemplo, na figura a seguir temos um digrafo, sua matriz e pesos e a matriz com todas as distâncias entre os pontos.

(**Dica:** *Algoritmo de Floyd-Warshall*).




Pesos

| | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| 1 | 0 | 6 | 8 | $\infty$ | 3 |
| 2 | $\infty$ | 0 | $\infty$ | 1 | 7 |
| 3 | $\infty$ | 4 | 0 | $\infty$ | $\infty$ |
| 4 | 2 | $\infty$ | 5 | 0 | $\infty$ |
| 5 | $\infty$ | $\infty$ | $\infty$ | 3 | 0 |

Distâncias

| | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| 1 | 0 | 6 | 8 | 6 | 3 |
| 2 | 3 | 0 | 6 | 1 | 6 |
| 3 | 7 | 4 | 0 | 5 | 10 |
| 4 | 2 | 8 | 5 | 0 | 5 |
| 5 | 5 | 11 | 8 | 3 | 0 |

23. Dadas duas sequências $A$ e $B$ contendo, respectivamente, $n$ valores reais positivos e a $n - 1$ operadores em $\{+, -, \times, /\}$, de mesma precedência. As sequências representam uma expressão matemática. Por exemplo: $A = \langle 0.3, 1.0, 4.0, 0.7, 0.2 \rangle$ e $B = \langle +, \times, -, \times, \rangle$ correspondem a $0.3 + 1.0 \times 4.0 - 0.7 \times 0.2$. Proponha um algoritmo eficiente para colocar parênteses na expressão de modo que o seu valor seja máximo. No exemplo, esse valor é 5.06 e pode ser obtido por $((0.3 + 1.0) \times 4.0) - (0.7 \times 0.2)$.

## Modelos das Soluções

1. i. L[i,j]: Maior subsequência comum de X[0...i-1] e Y[0...j-1]

   ii. $$L[i,j] = \begin{cases} 0, \text{ se } i=0 \text{ ou } j=0; \\ 1 + L[i-1,j-1], \text{ se } i>0,\ j>0 \text{ e } X[i-1]=Y[j-1]; \\ \max(L[i,j-1],L[i-1,j]), \text{ se } i>0,\ j>0 \text{ e } X[i-1]\neq Y[j-1]. \end{cases}$$

   iii. Calcular L[m,n].

2. i. P[i,j]: Maior subsequência palíndromo de S[i...j]

   ii. $$P[i,j] = \begin{cases} 0, \text{ se } i>j; \\ 1, \text{ se } i=j; \\ 2 + P[i+1,j-1], \text{ se } i<j \text{ e } S[i]=S[j]; \\ \max(P[i,j-1],P[i+1,j]), \text{ se } i<j \text{ e } S[i]\neq S[j]. \end{cases}$$

   iii. Calcular P[1,n].

3. i. R[i]: Menor penalização até o km $a_i$

   ii. $$R[i] = \begin{cases} 0, \text{ se } i=0; \\ \min\{R[j]+|200-(A[i]-A[j])|:\ 0\leq j<i\}; \end{cases}$$

   iii. Calcular R[n].

4. i. S[i] : maior soma de elementos de A[0...i], incluindo o A[i].

   ii. $$S[i] = \begin{cases} A[i], \text{ se } i=0 \text{ ou } S[i-1] \leq 0; \\ S[i-1]+A[i], \text{ se } i>0 \text{ e } S[i-1] > 0. \end{cases}$$

   iii. $\max\{S[i]:\ 0 \leq i \leq n-1\}$.

5. i. C[i,j] : menor custo para fazer os cortes necessários entre as marcas i e j.

   ii. $$C[i,j] = \begin{cases} 0, \text{ se } j-i\leq 1 \text{ ou se não há ponto de corte entre } i \text{ e } j; \\ \min\{C[i,Pk]+C[Pk,j]:\ i<Pk<j\} + (j-i), \text{ caso contrário.} \end{cases}$$

   iii. Calcular C[0,l].

6. i. L[i] : maior lucro obtido a partir de uma madeira de tamanho i.
   dados os tamanhos A[1...k] e os valores B[1...k].
   V[i]: maior valor de um pedaço de madeira de tamanho i, caso exista, ou 0.

   ii. $$L[i] = \begin{cases} 0, \text{ se } i = 0; \\ \max\{L[j]+L[i-j]:\ 1\leq j\leq i\}, \text{ se } i>0. \end{cases}$$

   iii. Calcular L[n].

7. i. S[i]: maior subseq. crescente de A[0...i], incluindo A[i].

   ii. $$S[i] = \begin{cases} 1, \text{ se } i=0; \\ 1, \text{ se } i>0 \text{ e não há valor menor que } A[i] \text{ antes dele} \\ \max\{1+S[k]:0\leq k\leq i \text{ e } A[k]<A[i]\}, \text{ caso contrário.} \end{cases}$$

   iii. Calcular o maior valor de S.

8. i. E[i,j]: distância de edição entre A[0...i-1] e B[0...j-1].
   diff(X,Y) = 1, se $X \neq Y$; e 0, caso contrário

   ii. $$E[i,j] = \begin{cases} i+j, \text{ se } i=0 \text{ ou } j=0; \\ \min\{E[i-1,j]+1,E[i,j-1]+1,E[i-1,j-1]+diff(A[i-1],B[j-1])\}. \end{cases}$$

   iii. Calcular E[m,n].

9. Dada uma sequência de tamanhos de carros C[$c_0, \ldots, c_{n-1}$] e o tamanho das pistas L.

   i. M[k,i,j]: maior numero de carros da fila C[k...n-1] que podem ser colocados na balsa - pista 1 com tam. i, e 2 de tam. j.

   ii. $$\text{M[k,i,j]} = \begin{cases} \text{0, se k}\geq\text{n, ou C[k]>i e C[k]>j;} \\ \text{1 + M[k+1,i-C[k],j], se k<n e i}\geq\text{C[k]>j;} \\ \text{1 + M[k+1,i,j-C[k]], se k<n e i<C[k]}\leq\text{j;} \\ \text{1 + max(M[k+1,i-C[k],j],M[k+1,i,j-C[k]]), caso contrário.} \end{cases}$$

   iii. Calcular M[0,L,L].

10. Dada uma sequência de palavras W[0,...,n-1] na ordem inversa à de impressão.

   i. D[i]: defeito mínimo do parágrafo formado pelas palavras w1...wi.
   C(m,k) = Tm+1+...+1+Tk. (Obs.: C(m,k) é curto se for no máximo L.)

   ii. $$\text{D[i]} = \begin{cases} \text{0, se C(i,0)}\leq\text{L;} \\ \text{min}\{(\text{L-C(i,k)})^3 \text{ + D[k-1]: 0<k}\leq\text{i e C(i,k)}\leq\text{L}\}. \end{cases}$$

   iii. Calcular D[n-1].

11. Sejam um conjunto de inteiros A[1..n], e T a soma dos elementos de A.
   Se T for múltiplo de 3, faça s=T/3; senão, retornar 0.

   i. M[i,j,k]: 1, se existem subconjuntos de A[1...i] cujas somas são j e k.

   ii. $$\text{M[i,j,k]} = \begin{cases} \text{1, se i=0, j=0 e k=0;} \\ \text{0, se i=0, e j>0 ou k>0;} \\ (\text{A}_i \leq\text{j} \cap \text{M[i-1,j-A}_i\text{,k]}) \cup (\text{A}_i \leq\text{k} \cap \text{M[i-1,j,k-A}_i\text{]}) \cup (\text{M[i-1,j,k]}). \end{cases}$$

   iii. Calcular M[n,s,s].

12. Sejam H[0,...,n-1] a sequência de alturas das torres de cubos.

   i. E[i] a altura da maior escada a partir da esquerda, e
   D[i] a altura da maior escada a partir da direita, ambas incluindo a torre i.

   ii. $$\text{E[i]} = \begin{cases} \text{1, se i=0;} \\ \text{min(E[i-1]+1,H[i]), para 0<i}\leq\text{n-1.} \end{cases}$$
   $$\text{D[i]} = \begin{cases} \text{1, se i=n-1;} \\ \text{min(D[i+1]+1,H[i]), para 0}\geq\text{i<n-1.} \end{cases}$$

   iii. Calcular max{min(E[i],D[i]):0≤i≤n-1}

13. Dada a sequência de valores de ações $V[1 \ldots n]$ e o valor da corretora $C$.

   i. L[i]: maior lucro no período $0 \ldots i$.

   ii. $$\text{L[i]} = \begin{cases} \text{0, se i} \leq \text{1;} \\ \text{max}\{L[k-1] \ + \ \text{max}(0, V[i] - V[k] - C) \mid 1 \leq k \leq i\} \end{cases}$$

   iii. calcular L[n].

14. i. V[i]: maior valor, não superior a $i$, que pode ser pago com as moedas $m_1 \ldots m_n$.

   ii. $$\text{V[i]} = \begin{cases} \text{0, se não existe moeda com valor menor ou igual a i;} \\ \text{max}\{m_k + V[i - M_k] \mid m_k \leq i, 1 \leq k \leq n\} \end{cases}$$

   iii. verificar se V[T] = T.

15. i. `S[i,j] : maior soma de elementos de A[1...i] não superior a j.`

ii. $$\texttt{S[i,j]} = \begin{cases} \texttt{0, se i=0 ou j=0;} \\ \texttt{S[i-1,j], se i>0, j>0 e A[i]>j;} \\ \texttt{max(S[i-1,j],A[i]+S[i-1,j-A[i]]), se i>0, j>0 e A[i]}\leq\texttt{j.} \end{cases}$$

iii. `Verificar se S[n,T]=T.`

16. i. `Dada uma sequência de` $n$ `cartas` $\langle s_0 \dots s_{n-1} \rangle$
`M[i,j]: maior diferença que se pode obter com as cartas` $s_i \dots s_j$.

ii. $$\texttt{M[i,j]} = \begin{cases} \texttt{0, se i > j;} \\ s_i\texttt{, se i = j;} \\ \texttt{max(}s_i - M[i+1,j],\ \ s_j - M[i,j-1]\texttt{), se i < j;} \end{cases}$$

iii. calcular M[0,n-1]

17. i. `Dada uma quantidade` $n$ `de pedras`
`W[i]: diz se é possível o jogador atual ganhar, com` $i$ `pedras na mesa.`

ii. $$\texttt{W[i]} = \begin{cases} \texttt{1, se (}i \geq 1 \texttt{ e } \neg W[i-1]\texttt{) ou (}i \geq 3 \texttt{ e } \neg W[i-3]\texttt{) ou (}i \geq 8 \texttt{ e } \neg W[i-8]\texttt{);} \\ \texttt{0, caso contrário;} \end{cases}$$

iii. calcular W[n]

18. i. `Dada a sequência de valores` $P[0 \dots n]$ `correspondente às ordens das matrizes.`
`Ordem da matriz` $A_i = P_i \times P_{i+1}$, `para` $0 \leq i \leq n-1$.
`M[i,j]: menor número de multiplicações escalares de` $A_i \times \dots \times A_j$.

ii. $$\texttt{M[i,j]} = \begin{cases} \texttt{0, se i = j;} \\ \texttt{min}\{M[i,k] + M[k+1,j] + P_i \cdot P_{k+1} \cdot P_{j+1} \mid i \leq k < j\} \end{cases}$$

iii. calcular M[0,n-1].

19. `Considere um vetor` $S[1..n]$, `onde` $S[1] = A[1]$ `e, para` $i > 0$, $S[i] = S[i-1] + A[i]$
`Se` $S[i] = 0$, `então` $A[1..i]$ `é uma sequência nula`
`Se, para` $i < j$, $S[i] = S[j]$, `então` $A[i+1..j]$ `é uma sequência nula`

20. Proceder de forma semelhante à questão anterior, considerando a soma módulo 3.

21. i. `Dada a largura n. A qtd. máxima de peças que cabe em uma fileira é q = n/10.`
`Consideremos que as peças são de larguras 1, 2 e 3.`
`F[i]: quantidade de fileiras distintas de tamanho i que podem ser feitas.`

ii. $$\texttt{F[i]} = \begin{cases} \texttt{i, se i < 3;} \\ \texttt{4, se i = 3;} \\ \texttt{F[i-1] + F[i-2] + F[i-3], se i > 3;} \end{cases}$$

iii. calcular F[q].

22. i. `D[i,j,k]: menor distância de` $i$ `a` $j$ `passando pelos vértices` $[1 \dots k]$.
`W[i,j]: peso do arco` $ij$.

ii. $$\texttt{D[i,j,k]} = \begin{cases} \texttt{W[i,j], se k = 0;} \\ \texttt{mín}\{D[i,j,k-1], D[i,k,k-1] + D[k,j,k-1]\}\texttt{, se k > 0;} \end{cases}$$